

Patrón Strategy

Nombre del autor

 **Tiempo estimado: 25min**

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
Laboratorio – Análisis y Diseño de Sistemas 2

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Adaptabilidad	El compromiso permite al estudiante asumir con seriedad el diseño de soluciones eficientes y sostenibles, entendiendo que la correcta aplicación de patrones y principios de diseño impacta directamente en la calidad del software. Diseñar bien no solo es una habilidad técnica, sino una muestra de responsabilidad profesional con los usuarios, el equipo y el producto final.

1.2 Competencia(s)

Tipo de Competencia	
Competencia General	La adaptabilidad es el valor clave para dominar los patrones de comportamiento. Estos patrones buscan desacoplar y flexibilizar el código, haciendo que el sistema pueda cambiar y evolucionar sin grandes modificaciones.

2. Palabras Clave

Patrón Strategy: Es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos.

3. Introducción

El patrón Strategy es una solución de diseño que permite organizar distintos algoritmos dentro de clases separadas, de manera que puedan intercambiarse fácilmente según la necesidad del programa. Su principal ventaja es que ayuda a que el código sea más flexible, ordenado y fácil de mantener cuando existen varias formas de realizar una misma tarea.

4. Contenidos

Patrón Strategy

Strategy es un patrón de diseño de comportamiento que te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables.

Por ejemplo un día decidiste crear una aplicación de navegación para viajeros ocasionales. La aplicación giraba alrededor de un bonito mapa que ayudaba a los usuarios a orientarse rápidamente en cualquier ciudad.

Una de las funciones más solicitadas para la aplicación era la planificación automática de rutas. Un usuario debía poder introducir una dirección y ver la ruta más rápida a ese destino mostrado en el mapa.

La primera versión de la aplicación sólo podía generar las rutas sobre carreteras. Las personas que viajaban en coche estaban locas de alegría. Pero, aparentemente, no a todo el mundo le gusta conducir durante sus vacaciones. De modo que, en la siguiente actualización, añadiste una opción para crear rutas a pie. Después, añadiste otra opción para permitir a las personas utilizar el transporte público en sus rutas.

Sin embargo, esto era sólo el principio. Más tarde planeaste añadir la generación de rutas para ciclistas, y más tarde, otra opción para trazar rutas por todas las atracciones turísticas de una ciudad.

Aunque desde una perspectiva comercial la aplicación era un éxito, la parte técnica provocaba muchos dolores de cabeza. Cada vez que añadías un nuevo algoritmo de enrutamiento, la clase principal del navegador doblaba su tamaño. En cierto momento, la bestia se volvió demasiado difícil de mantener.

Cualquier cambio en alguno de los algoritmos, ya fuera un sencillo arreglo de un error o un ligero ajuste de la representación de la calle, afectaba a toda la clase, aumentando las probabilidades de crear un error en un código ya funcional.

Además, el trabajo en equipo se volvió ineficiente. Tus compañeros, contratados tras el exitoso lanzamiento, se quejaban de que dedicaban demasiado tiempo a resolver conflictos de integración. Implementar una nueva función te exige cambiar la misma clase enorme, entrando en conflicto con el código producido por otras personas.

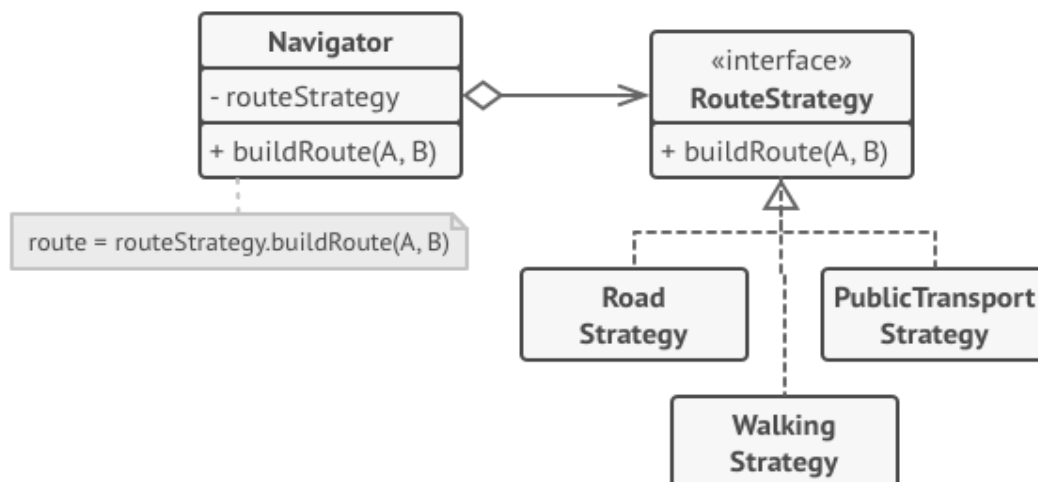
El patrón Strategy sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas estrategias.

La clase original, llamada contexto, debe tener un campo para almacenar una referencia a una de las estrategias. El contexto delega el trabajo a un objeto de estrategia vinculado en lugar de ejecutarlo por su cuenta.

La clase contexto no es responsable de seleccionar un algoritmo adecuado para la tarea. En lugar de eso, el cliente pasa la estrategia deseada a la clase contexto. De hecho, la clase contexto no sabe mucho acerca de las estrategias. Funciona con todas las estrategias a través de la misma interfaz genérica, que sólo expone un único método para disparar el algoritmo encapsulado dentro de la estrategia seleccionada.

De esta forma, el contexto se vuelve independiente de las estrategias concretas, así que puedes añadir nuevos algoritmos o modificar los existentes sin cambiar el código de la clase contexto o de otras estrategias.

Figura 1 – UML de estrategia de planificación de rutas usando el patrón strategy.



En nuestra aplicación de navegación, cada algoritmo de enrutamiento puede extraerse y ponerse en su propia clase con un único método *crearRuta*. El método acepta un origen y un destino y devuelve una colección de puntos de control de la ruta.

Incluso contando con los mismos argumentos, cada clase de enrutamiento puede crear una ruta diferente. A la clase navegadora principal no le importa qué algoritmo se selecciona ya que su labor principal es representar un grupo de puntos de control en el mapa. La clase tiene un método para cambiar la estrategia activa de enrutamiento, de modo que sus clientes, como los botones en la interfaz de usuario, pueden sustituir el comportamiento seleccionado de enrutamiento por otro.

5. Conclusiones

El patrón strategy permite separar responsabilidades y evitar que una sola clase concentre demasiada lógica. Gracias a esto, es más sencillo agregar nuevos comportamientos, modificar los existentes y mantener el sistema sin afectar otras partes del programa.

6. Referencias

1. Refactoring.Guru. Strategy [Internet]. Refactoring.Guru; [citado 2026]. Disponible en: <https://refactoring.guru/es/design-patterns/strategy>